

分数背包

```
# 圣诞老人
n, w = map(int, input().split())
candies = []
for _ in range(n):
    p, q = map(int, input().split())
    for _ in range(q):
        candies.append(p / q)
candies.sort(reverse=True)
value = sum(candies[:w])
print("{:.1f}".format(value))
```

输入:

```
4 15
100 4
412 8
266 7
591 2
```

输出:

```
1193.0
```

0-1背包

```
def knapsack(weights, values, capacity):
    n = len(values) # 物品的数量
    # 创建一个二维数组 dp, 其中 dp[i][w] 表示前 i 个物品在容量为 w 的背包中能达到的最大价值。
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
    # 遍历每一个物品
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                # 如果当前物品可以放入背包, 则考虑两种情况:
                # 1. 不放入该物品, 此时的价值为 dp[i-1][w]
                # 2. 放入该物品, 此时的价值为 values[i-1] + dp[i-1][w-weights[i-1]]
                # 取两者中的较大值
                dp[i][w] = max(dp[i - 1][w], values[i - 1] + dp[i - 1][w -
weights[i - 1]])
            else:
                # 如果当前物品不能放入背包, 则其价值与不包含该物品时相同
                dp[i][w] = dp[i - 1][w]

    # dp[n][capacity] 即为所求的最大价值
    return dp[n][capacity]

# 示例
weights = [2, 3, 4, 5]
values = [3, 4, 5, 6]
capacity = 5
print(knapsack(weights, values, capacity))
```

```
#采药问题
#T 采药总时间, M 草药数目
```

```

T,M = map(int,input().split())
#dp[j]用于存储当采药总时间为j 时，由目前已给出的草药数据可采得的最大价值
dp = [0]*(T+1)
last_data = list(dp)
#外层循环遍历草药数据 (M)
for i in range(M):
    time,value = map(int,input().split())
    #内层循环遍历采药时间 (T)
    for j in range(1,T+1):
        if j >= time:
            dp[j] = max(last_data[j],value + last_data[j-time])
        else:
            dp[j] = last_data[j]
    #用last_data 将本次运算结果存起来以供下一次循环运算使用
    last_data = list(dp)
print(dp[-1])

```

输入：
 70 3
 71 100
 69 1
 1 2
 输出：
 3

年份calendar包

```

import calendar
print(calendar.isleap(2020)) # True判断闰年

```